國立陽明交通大學
NATIONAL YANG MING CHIAO TUNG UNIVERSITY

Institute of Artificial Intelligence Innovation
Department of Computer Science

Embedded Operating System

HW02: Embedded System Generic Driver

Shuo-Han Chen (陳碩漢　)
shch@nycu.edu.tw

Wed. 13:20 – 16:20 EDB06

Purpose for this Homework

- **In previous homework, you have learned**
  - How to write kernel module
  - How to read files from kernel module
- **Part 1. Extend kernel module into a character driver**
  - Need to support open/close/read/write
  - Implement above functions to access a backend file
- **Part 2. Write a C based program to access the driver**
  - Note that C program are in user space, Just like any other regular program you wrote
  - In Linux, you can write a makefile to compile you c program

Extend kernel module into a character driver

- **Refer to the slide of character driver, you should have**

```
static ssize_t demo_write(struct file *f, const char __user *buf,
                          size_t len, loff_t *off)
{
        [...]
}

static ssize_t demo_read(struct file *f, char __user *buf,
                         size_t len, loff_t *off)
{
        [...]
}

static struct file_operations demo_fops =
{
        .owner = THIS_MODULE,
        .read = acme_read,
        .write = acme_write
};
```

Two addition functions are required here for open & close

Add .open = … & .close = … here

Extend kernel module into a character driver

- **Then, extend the init & exit functions of kernel module as follows**

Major number (your driver identification number)

Minor number (The number of drivers that can be opened)

```
static dev_t demo_dev = MKDEV(202,128);
static struct cdev demo_cdev;

static int __init demo_init(void)
{
    register_chrdev_region(demo_dev, 1, \demo");
    cdev_init(&demo_cdev, &demo_fops);
    cdev_add(&demo_cdev, demo_dev, demo_count);
}

static void __exit demo_exit(void)
{
    cdev_del(&demo_cdev);
    unregister_chrdev_region(demo_dev, 1);
    iounmap(demo_buf);
}

module_init(demo_init);
module_exit(demo_exit);
```

Here is the name of your driver

Extend kernel module into a character driver

- **Implement above functions to access a backend file**
  - In the previous homework, you learn how to open a file in the kernel.
  - Now, treat the previous function as a user program interface, and allow it to access a backend file through your driver.
  - You need to create a file inside the kernel and use the relative path to access it.

Write a C based program to access the driver

- **After you insmod your driver, you should use following command to create a special file which represent your character driver**

Major number (your driver identification number)

```
sudo mknod /dev/demo c 202 0
```

index number (The number should below your minor number)

Character Driver

- **So in your C based program, it would look like this**

```
system("sudo mknod /dev/demo c 202 0");

fd = open("/dev/demo", O_RDWR);

ret = read(fd, buf, buffersize);

ret = write(fd, buf, buffersize);
```

- **Then, you need to**
  1. Write a string to the driver through write
  2. Read back the written string from the driver
  3. Out the read result to bash terminal

Submission

- Submit your PDF report and source code in a zip format to the google Form.
- The google form will be published in Teams before the Deadline.
- The zip folder structure should look like this:
  - HW02_studentID.zip
    - report.pdf
    - driver
      - demo.c
      - Makefile
    - user
      - main.c
      - Makefile

Not like this:

- HW02_studentID.zip
  - folder
    - report.pdf
    - driver
      - demo.c
      - Makefile
    - user
      - main.c
      - Makefile

Grading

- Part1 Extend kernel module into a character driver - 80%
- Part2 Write a C based program to access the driver - 20%
- If your report format or submission folder structure does not meet the requirements, 20% will be deducted from your total score. Please double-check before submitting.

Q & A

Remember to submit your report.

The deadline is set as 4/23 23:59

Late submission will be not accepted!